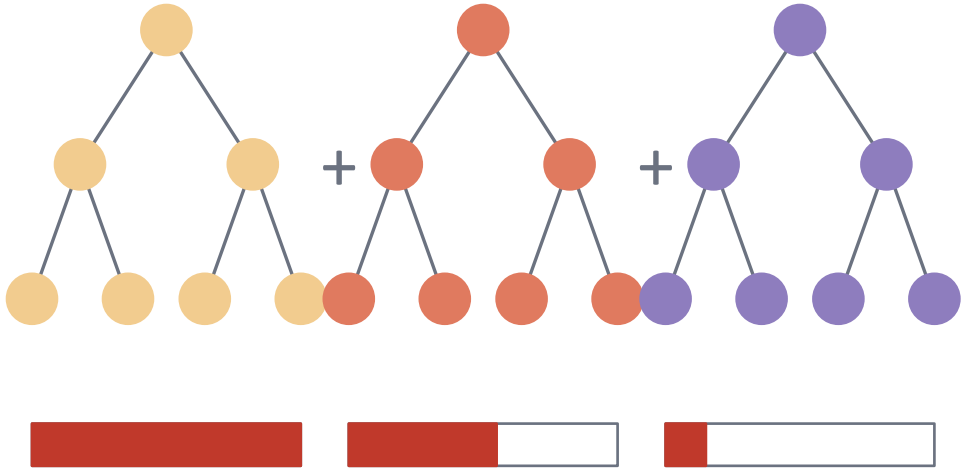
# Бустинг и его реализации

От разложения ошибки до XGBoost, LightGBM и CatBoost



ошибка уменьшается с каждым деревом

Елена Кантонистова

# План лекции

**01**

## Recap: разложение ошибки и бэггинг

смещение, разброс, шум; бутстрэп, случайный лес, OOB

**02**

## Бустинг

идея, бустинг в регрессии, пример с таблицей

**03**

## Градиентный бустинг

антиградиент, алгоритм, функции потерь, регуляризация

**04**

## Реализации

XGBoost, LightGBM, CatBoost: что внутри и как настраивать

# 01 Разложение ошибки и бэггинг

Почему модель ошибается и как ансамбли уменьшают разброс

# Почему модель ошибается?

Чтобы улучшить модель, нужно понять, откуда берётся ошибка в предсказаниях.

### Модель переобучена?

Хорошо на обучении, плохо на новых данных

### Модель плохо предсказывает целевую переменную?

Даже в среднем ответы далеки от правильных

### В самих данных много шума?

Часть ошибки нельзя устранить никакой моделью

Ответ даёт **разложение ошибки на смещение и разброс** (bias-variance decomposition).

# Разложение ошибки (bias-variance decomposition)

**Утверждение:** *ошибку модели $a(x)$ можно представить в виде*

$$\mathrm{Err}(x) = \mathrm{Bias}^2\big(a(x)\big) + \mathrm{Var}\big(a(x)\big) + \sigma^2$$

### Смещение (bias)

$\mathrm{Bias}(a(x))$ — средняя ошибка по всем возможным обучающим выборкам. Показывает, насколько хорошо модель в среднем предсказывает целевую переменную: большое смещение — плохое предсказание.

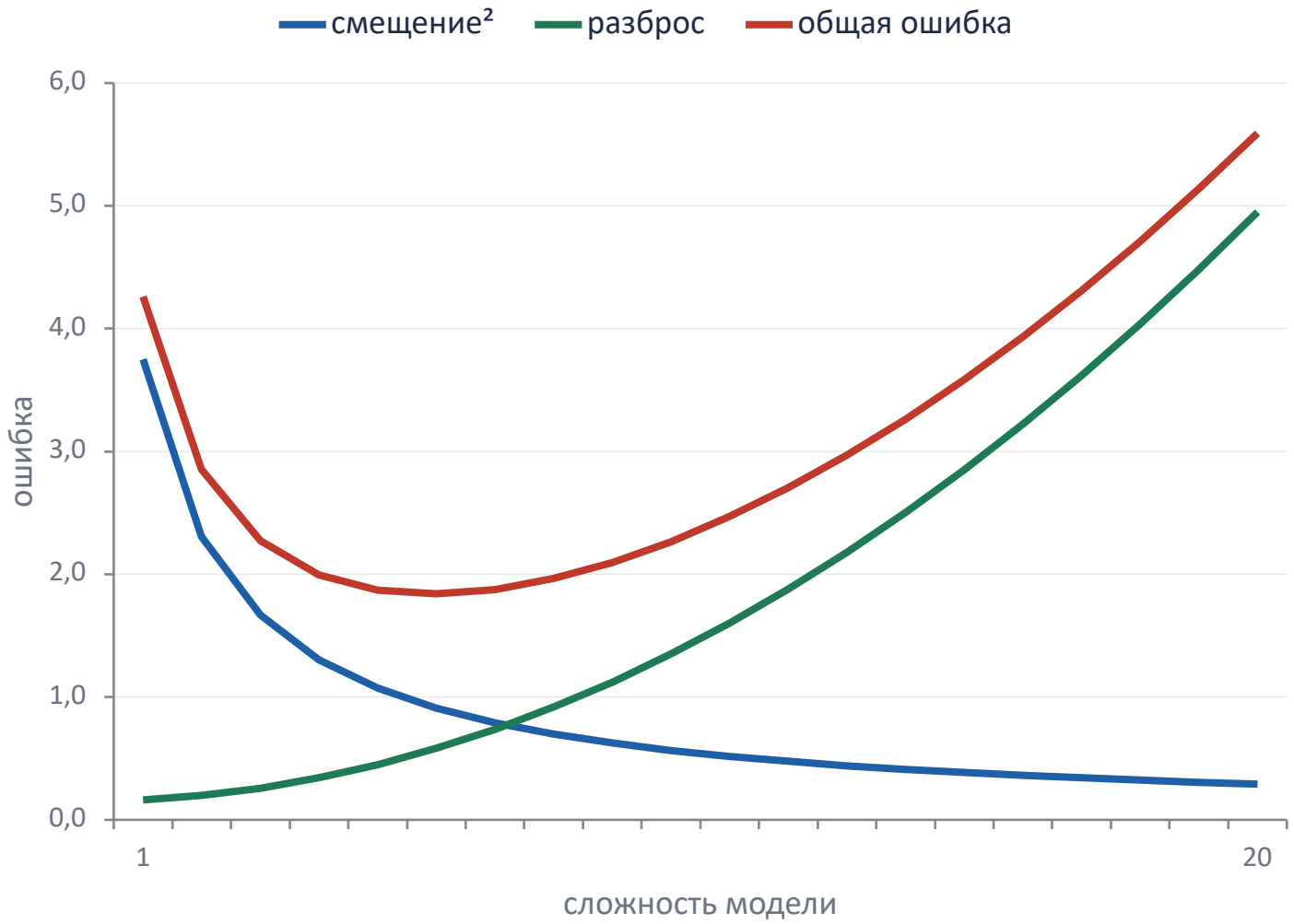
### Разброс (variance)

$\mathrm{Var}(a(x))$ — насколько сильно меняется ответ при обучении на разных выборках. Большой разброс — модель чувствительна к данным, т. е. сильно переобучена.

### Шум

$\sigma^2$ — неустранимая ошибка, заложенная в самих данных. Её не уменьшить выбором модели.

# Bias-variance tradeoff




**Недообучение**

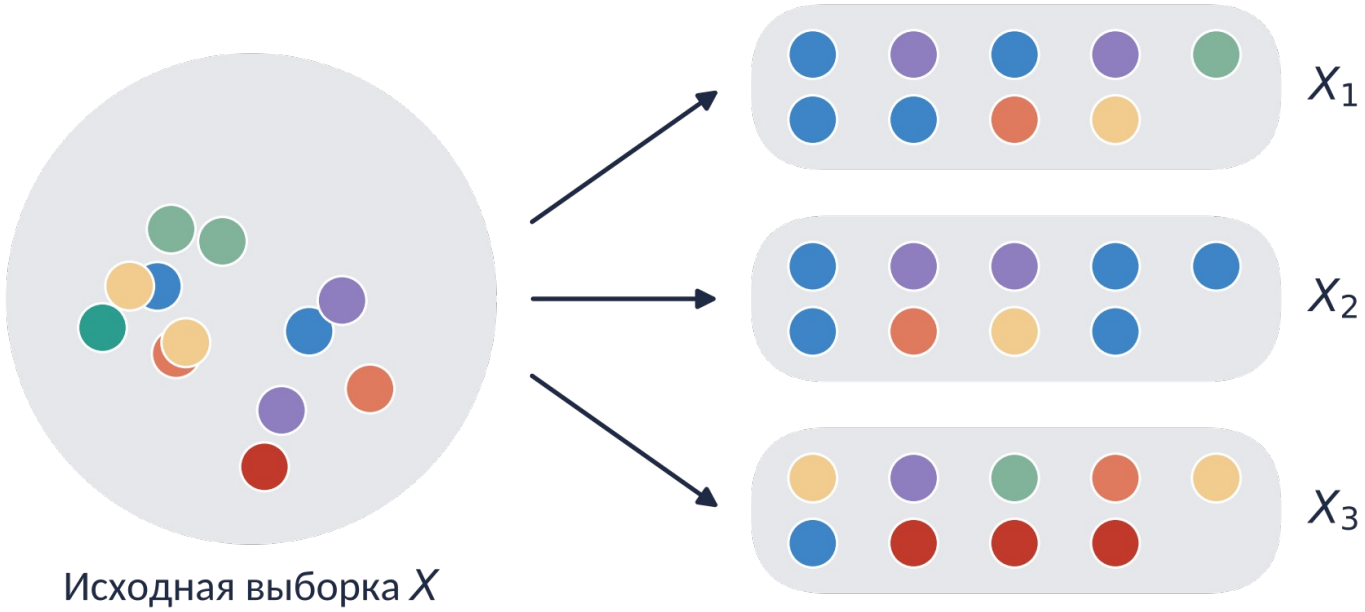
Простая модель: большое смещение, малый разброс

**Переобучение**

Сложная модель: малое смещение, большой разброс

# Бутстрэп

Дана выборка $X$ из $\ell$ объектов. **Бутстрэп:** равномерно выбираем из $X$ $\ell$ объектов с возвращением — в новой выборке будут повторы. Повторяем $N$ раз и получаем выборки $X_1, \ldots, X_N$.




В каждую бутстрэп-выборку попадает в среднем $\approx 63\%$ уникальных объектов исходной выборки.

# Бэггинг (bootstrap aggregation)

- С помощью бутстрэпа получаем выборки $X_1, \dots, X_N$
- Обучаем по каждой модель — базовые алгоритмы $b_1(x), \dots, b_N(x)$
- Итоговый ответ — среднее (в классификации — голосование):

$$a(x) = \frac{1}{N} \sum_{j=1}^{N} b_j(x)$$

## Утверждение

**1) Бэггинг не ухудшает смещение:** смещение композиции равно смещению одного базового алгоритма.

**2) Если базовые алгоритмы некоррелированы,** то дисперсия бэггинга в $N$ раз меньше дисперсии отдельного алгоритма.

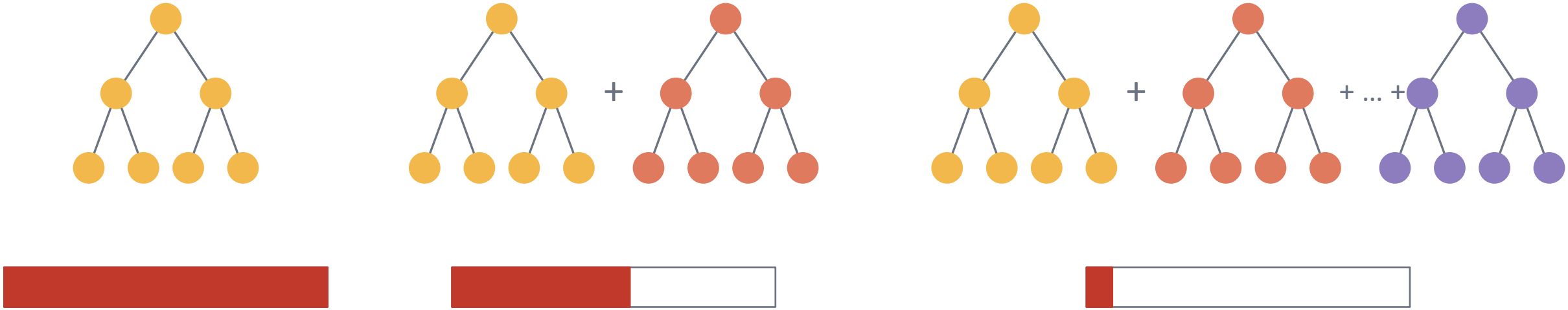
Вывод: бэггинг уменьшает разброс. Базовые модели должны быть сложными (малое смещение) и как можно менее похожими.

# 02 Бустинг

Каждый следующий алгоритм исправляет ошибки предыдущих

# Идея бустинга

**Идея:** *строим набор алгоритмов, каждый из которых исправляет ошибку предыдущих. Ответ — сумма всех алгоритмов.*




В отличие от бэггинга деревья строятся последовательно и зависят друг от друга.

# Бустинг в задаче регрессии

Решаем задачу регрессии с квадратичной функцией потерь:

$$\frac{1}{2}\sum_{i=1}^{\ell}\big(a(x_i)-y_i\big)^2 \to \min_a$$

Ищем алгоритм в виде суммы $N$ базовых алгоритмов из семейства $\mathcal{A}$:

$$a(x)=\sum_{n=1}^{N} b_n(x), \qquad b_n \in \mathcal{A}$$

**Шаг 1.** Ищем $b_1(x)$, минимизирующий ошибку:

$$b_1(x)=\arg\min_{b\in\mathcal{A}} \frac{1}{2}\sum_{i=1}^{\ell}\big(b(x_i)-y_i\big)^2$$

Ошибка на объекте:

$$s_i = y_i - b_1(x_i)$$

Следующий алгоритм настраиваем на эту ошибку: **целевая переменная для $b_2$ — вектор ошибок $s$, а не исходный $y$**

# Бустинг в задаче регрессии: шаги $2, \ldots, N$

**Шаг 2.** Ищем $b_2(x)$, настраивающийся на ошибки $s$ первого алгоритма:

$$b_2(x) = \arg\min_{b \in \mathcal{A}} \frac{1}{2} \sum_{i=1}^{\ell} \big(b(x_i) - s_i\big)^2$$

**Шаг N.** Каждый следующий алгоритм настраиваем на ошибку уже построенной композиции:

$$s_i^{(N)} = y_i - \sum_{n=1}^{N-1} b_n(x_i) = y_i - a_{N-1}(x_i)$$

$$b_N(x) = \arg\min_{b \in \mathcal{A}} \frac{1}{2} \sum_{i=1}^{\ell} \Big(b(x_i) - s_i^{(N)}\Big)^2$$

# Бустинг: пример. Шаг 1

| $y$ (истина) | $b_1$ (дерево 1) |
|---|---|
| 100 | **90** |
| 150 | **140** |
| 200 | **210** |
| 120 | **130** |
| 180 | **170** |

**Первое дерево приближает целевую переменную**

Предсказываем цену: первое дерево $b_1$ ошибается на каждом объекте примерно на $\pm 10$.

# Бустинг: пример. Ошибки первого дерева

| $y$ (истина) | $b_1$ (дерево 1) | $s_1 = y - b_1$ |
|---|---|---|
| 100 | 90 | **10** |
| 150 | 140 | **10** |
| 200 | 210 | **−10** |
| 120 | 130 | **−10** |
| 180 | 170 | **10** |

**Считаем остатки $s_1 = y - b_1$**

Эти остатки становятся новой целевой переменной: второе дерево будет учиться предсказывать именно их.

# Бустинг: пример. Второе дерево

| $y$ (истина) | $b_1$ (дерево 1) | $s_1 = y - b_1$ | $b_2$ (дерево 2) | $a = b_1 + b_2$ |
|---|---|---|---|---|
| 100 | 90 | 10 | **8** | **98** |
| 150 | 140 | 10 | **9** | **149** |
| 200 | 210 | −10 | **–8** | **202** |
| 120 | 130 | −10 | **–7** | **123** |
| 180 | 170 | 10 | **7** | **177** |

**Дерево $b_2$ предсказывает остатки, складываем ответы**

$a = b_1 + b_2$ уже намного ближе к истине: ошибка на каждом объекте не больше 3.

# Бустинг: пример. Третье дерево

| $y$ (истина) | $b_1$ (дерево 1) | $s_1 = y - b_1$ | $b_2$ (дерево 2) | $a = b_1 + b_2$ | $s_2 = y - a$ | $b_3$ (дерево 3) |
|---|---|---|---|---|---|---|
| 100 | 90 | 10 | 8 | 98 | **2** | **1** |
| 150 | 140 | 10 | 9 | 149 | **1** | **0** |
| 200 | 210 | −10 | −8 | 202 | **−2** | **0** |
| 120 | 130 | −10 | −7 | 123 | **−3** | **−1** |
| 180 | 170 | 10 | 7 | 177 | **3** | **2** |

**Новые остатки** $s_2 = y - (b_1 + b_2)$

Третье дерево $b_3$ обучается на $s_2$ — уточняет то, что не исправили первые два.

# Бустинг: пример. Итоговый прогноз

| $y$ (истина) | $b_1$ (дерево 1) | $s_1 = y - b_1$ | $b_2$ (дерево 2) | $a = b_1 + b_2$ | $s_2 = y - a$ | $b_3$ (дерево 3) | $a = b_1 + b_2 + b_3$ |
|---|---|---|---|---|---|---|---|
| 100 | 90 | 10 | 8 | 98 | 2 | 1 | **99** |
| 150 | 140 | 10 | 9 | 149 | 1 | 0 | **149** |
| 200 | 210 | −10 | −8 | 202 | −2 | 0 | **202** |
| 120 | 130 | −10 | −7 | 123 | −3 | −1 | **122** |
| 180 | 170 | 10 | 7 | 177 | 3 | 2 | **179** |

**100**

MSE после $b_1$

**5.4**

MSE после $b_1 + b_2$

**2.2**

MSE после $b_1 + b_2 + b_3$

# Смещение и разброс бустинга

### Бустинг

Целенаправленно уменьшает ошибку — смещение маленькое. Но итоговый алгоритм сложный, поэтому разброс может быть большим.

### Бэггинг / случайный лес

Не меняет смещение, но уменьшает разброс за счёт усреднения независимых моделей.

**Вывод.** Для бэггинга берём глубокие деревья (малое смещение), а для бустинга — неглубокие деревья, **глубины 3–6**, чтобы не переобучиться.

# 03 Градиентный бустинг

Обобщаем бустинг на произвольную дифференцируемую функцию потерь

# Остатки — это антиградиент

В регрессии с MSE на шаге $N$ мы обучаем $b_N$ на остатки уже построенной композиции.

**Утверждение.** *Ошибка на $N$-м шаге — это антиградиент функции потерь по ответу модели, вычисленный в точке ответа уже построенной композиции:*

$$s_i^{(N)} = y_i - a_{N-1}(x_i) = -\frac{\partial}{\partial z}\frac{1}{2}(z - y_i)^2\Big|_{z=a_{N-1}(x_i)}$$

- Значит, бустинг — это **градиентный спуск**, где шаг делается не по параметрам, а по ответам модели
- Заменим MSE на любую дифференцируемую $L(y, z)$ — получим **градиентный бустинг**

# Градиентный бустинг

Пусть $L(y, z)$ — произвольная дифференцируемая функция потерь. Строим алгоритм вида

$$a_N(x) = \sum_{n=1}^{N} \gamma_n b_n(x)$$

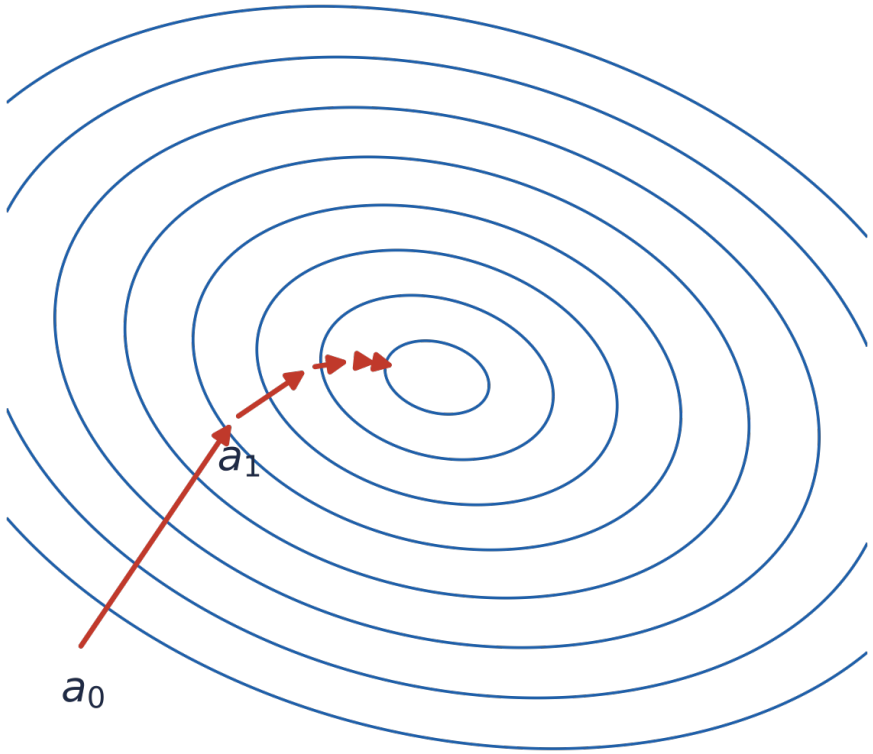
где на $N$-м шаге базовый алгоритм приближает антиградиент:

$$b_N(x) = \arg\min_{b \in \mathcal{A}} \sum_{i=1}^{\ell} \left( b(x_i) - s_i^{(N)} \right)^2 \qquad s_i^{(N)} = -\frac{\partial L(y_i, z)}{\partial z}\Big|_{z=a_{N-1}(x_i)}$$

Коэффициент $\gamma_N$ подбираем так, чтобы минимизировать ошибку:

$$\gamma_N = \arg\min_{\gamma \in \mathbb{R}} \sum_{i=1}^{\ell} L\big(y_i,\ a_{N-1}(x_i) + \gamma\, b_N(x_i)\big)$$

# Градиентный спуск в пространстве функций




Обычный градиентный спуск — шаг по параметрам:

$$w^{(t)} = w^{(t-1)} - \eta \nabla_w Q\left(w^{(t-1)}\right)$$

Градиентный бустинг — шаг по вектору ответов на объектах обучения:

$$a_N(x_i) = a_{N-1}(x_i) + \gamma_N b_N(x_i)$$

$$b_N(x_i) \approx -\frac{\partial L(y_i, z)}{\partial z}\Big|_{z=a_{N-1}(x_i)}$$

# Алгоритм градиентного бустинга

1: Инициализация константой: $a_0(x) = \arg\min_{c \in \mathbb{R}} \sum_{i=1}^{\ell} L(y_i, c)$

2: **для** $N = 1, \ldots, N_{\max}$:

3: антиградиент: $s_i^{(N)} = -\dfrac{\partial L(y_i, z)}{\partial z}\Big|_{z = a_{N-1}(x_i)}, \quad i = 1, \ldots, \ell$

4: обучаем дерево $b_N$ на выборке $\{(x_i,\ s_i^{(N)})\}_{i=1}^{\ell}$ по MSE

5: шаг: $\gamma_N = \arg\min_{\gamma} \sum_i L\big(y_i,\ a_{N-1}(x_i) + \gamma\, b_N(x_i)\big)$

6: обновление: $a_N(x) = a_{N-1}(x) + \eta\, \gamma_N\, b_N(x)$

7: вернуть $a_{N_{\max}}(x)$

# Функции потерь и их антиградиенты

| Задача | Функция потерь $L(y, z)$ | Антиградиент $s$ |
|---|---|---|
| Регрессия, MSE | $\frac{1}{2}(y-z)^2$ | $y-z$ |
| Регрессия, MAE | $\lvert y-z\rvert$ | $\text{sign}(y-z)$ |
| Классификация, logloss, $y \in \{0,1\}$ | $-y \log \sigma(z) - (1-y) \log\big(1-\sigma(z)\big)$ | $y-\sigma(z)$ |
| Классификация, экспоненциальная (AdaBoost) | $e^{-yz},\ \ y \in \{-1,+1\}$ | $y\, e^{-yz}$ |

Антиградиент $s = -\partial L / \partial z$. Для logloss $z$ — «сырой» выход модели (логит), $\sigma(z)$ — сигмоида. Для MAE антиградиент — только знак остатка: модель устойчивее к выбросам.

# Выбор базовых алгоритмов

### Слишком простые

Плохо приближают антиградиент функции потерь — градиентный бустинг может свестись к случайному блужданию.

### Слишком сложные

За несколько шагов бустинг подгонится под обучающую выборку — получим переобученный алгоритм.

**Чаще всего базовые алгоритмы — решающие деревья.** Они не должны быть ни очень маленькими, ни очень глубокими: оптимальная глубина — от 3 до 6 (зависит от задачи).

# Регуляризация: сокращение шага и подвыборки

**Сокращение шага (shrinkage).** Каждый базовый алгоритм добавляем с небольшим весом:

$$a_N(x) = a_{N-1}(x) + \eta\,\gamma_N\,b_N(x), \qquad \eta \in (0;1]$$

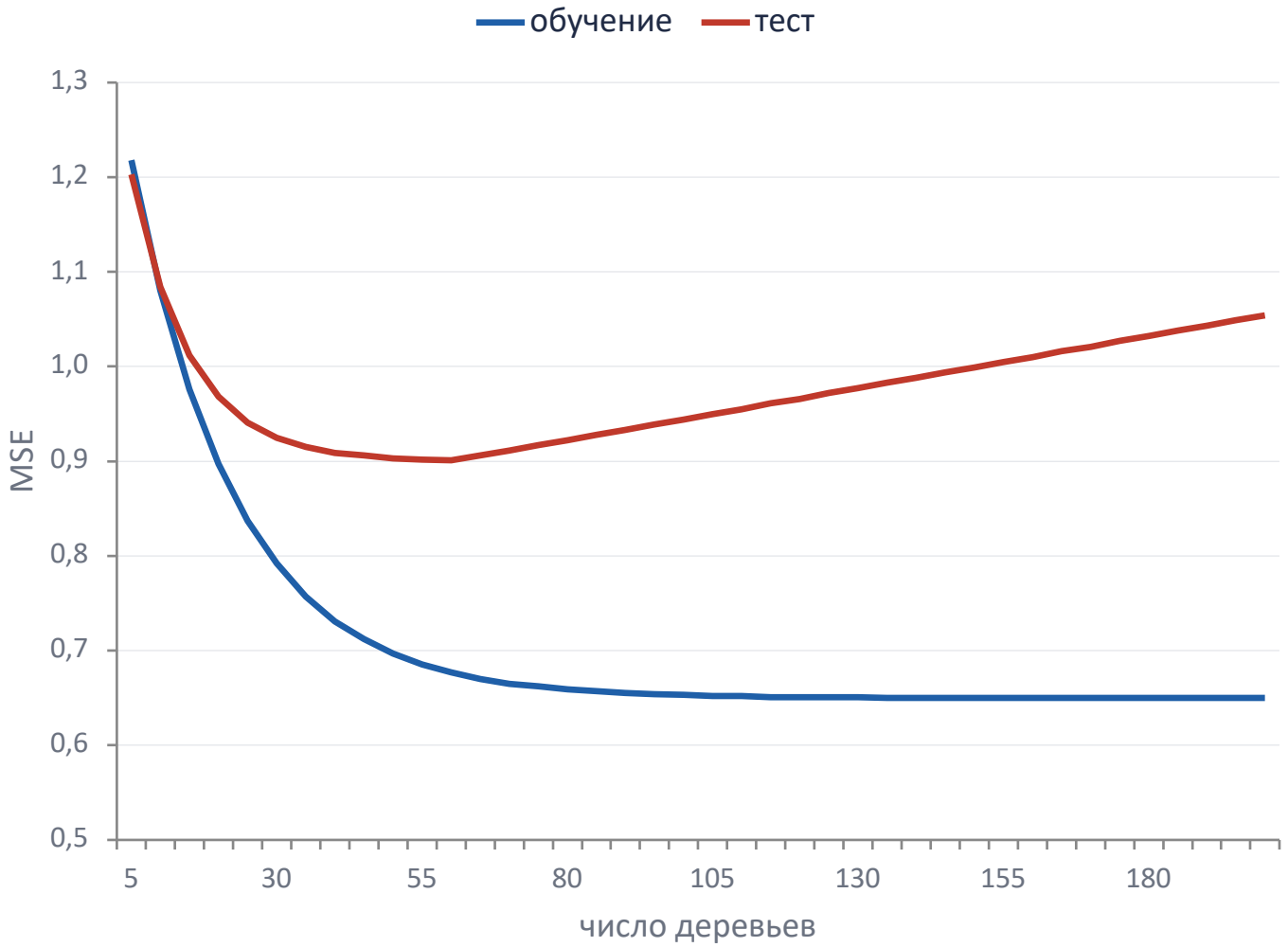
Чем меньше темп обучения $\eta$, тем меньше доверие к каждому дереву и тем лучше качество итоговой композиции — но тем больше деревьев нужно.

**Стохастический градиентный бустинг.** Обучаем $b_N$ не по всей выборке $X$, а по случайной подвыборке $X^k \subset X$ (обычно $|X^k| \approx \frac{1}{2}|X|$).

+ снижается влияние шума в данных
+ вычисления становятся быстрее

В реализациях: subsample / bagging_fraction — по объектам, colsample_bytree / feature_fraction — по признакам

# Количество итераций бустинга




*На каждом шаге ошибка на обучении целенаправленно уменьшается. Если процесс не остановить, достигнем нулевой ошибки — и переобучимся.*

**Ранняя остановка**

Следим за качеством на валидации и останавливаемся, когда оно перестаёт улучшаться (early_stopping_rounds).

# 04 Реализации градиентного бустинга

XGBoost, LightGBM, CatBoost

# XGBoost, LightGBM, CatBoost




### XGBoost

исследовательский проект Tianqi Chen; широкая известность и статья на KDD — 2016

### LightGBM

Microsoft: открытый код в 2016, стабильная версия и статья на NeurIPS — 2017

### CatBoost

Яндекс открывает исходный код; статья на NeurIPS — 2018

github.com/dmlc/xgboost github.com/microsoft/LightGBM github.com/catboost/catboost

# Общее и различия

Все три — градиентный бустинг над деревьями. Отличаются тем, как строят дерево и как работают с данными.

### XGBoost

Вторая производная функции потерь и явная регуляризация в функционале; критерий разбиения выводится из оптимальных значений в листьях

### LightGBM

Скорость: рост дерева по листьям, гистограммы, GOSS (отбор объектов), EFB (склейка разреженных признаков)

### CatBoost

Категориальные признаки «из коробки»: ordered target statistics, комбинации признаков; симметричные деревья и ordered boosting

# XGBoost (eXtreme Gradient Boosting)

На каждом шаге градиентного бустинга решается задача (раскрыли квадрат, выбросили константу):

$$\sum_{i=1}^{\ell} \big(b(x_i) - s_i\big)^2 \to \min_b \quad \Longleftrightarrow \quad \sum_{i=1}^{\ell} \Big( - s_i\, b(x_i) + \frac{1}{2}\, b^2(x_i)\Big) \to \min_b$$

На каждом шаге XGBoost решается задача:

$$\sum_{i=1}^{\ell} \Big( - s_i\, b(x_i) + \frac{1}{2}\, h_i\, b^2(x_i)\Big) + \gamma J + \frac{\lambda}{2} \sum_{j=1}^{J} b_j^2 \ \to \ \min_b$$

$$h_i = \frac{\partial^2 L(y_i, z)}{\partial z^2}\Big|_{z=a_{N-1}(x_i)}$$

$\gamma J$ — штраф за число листьев $J$; $\frac{\lambda}{2}\sum_j b_j^2$ — штраф за величину значений в листьях

# XGBoost: откуда берётся $h_i$

Разложим функцию потерь в ряд Тейлора до второго порядка в точке текущего ответа композиции:

$$L\big(y_i,\, a_{N-1}(x_i) + b(x_i)\big) \approx L\big(y_i,\, a_{N-1}(x_i)\big) - s_i\, b(x_i) + \frac{1}{2}\, h_i\, b^2(x_i)$$

### Обычный GB

Использует только первую производную: дерево приближает антиградиент $s_i$ — это аналог шага градиентного спуска.

### XGBoost

Учитывает и вторую производную $h_i$ — аналог шага метода Ньютона: шаг автоматически масштабируется по кривизне функции потерь.

# XGBoost: значения в листьях

Дерево в каждом листе $R_j$ отвечает константой $b_j$. Поэтому функционал распадается на сумму по листьям, и в каждом листе получается обычная парабола:

$$-S_j\, b_j + \frac{1}{2}\,(H_j + \lambda)\, b_j^2 \;\to\; \min_{b_j}$$

$S_j$ — сумма антиградиентов $s_i$ по объектам листа, $H_j$ — сумма вторых производных $h_i$

**Значение в листе:**

$$b_j^* = \frac{S_j}{H_j + \lambda}$$

## Интуиция на MSE

Для MSE $h_i = 1$, поэтому $H_j = n_j$ — число объектов в листе, а $b_j^*$ — сумма остатков в листе, делённая на $n_j + \lambda$. Это средний остаток, «поджатый» к нулю: чем меньше лист, тем сильнее.

# XGBoost: как выбрать разбиение и когда остановиться

**1** Считаем «качество» листа: оно большое, когда объекты листа дружно тянут ответ в одну сторону

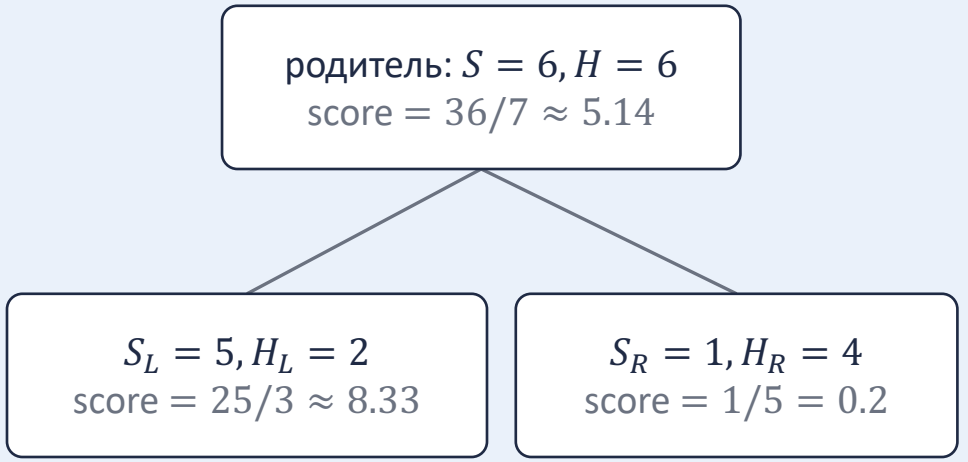
$$\text{score} = \frac{S^2}{H + \lambda}$$

**2** Делим, если дети вместе лучше родителя:

$$\text{Gain} = \frac{1}{2}\left(\text{score}_L + \text{score}_R - \text{score}_{\text{родитель}}\right) - \gamma$$

**3** Перебираем признаки и пороги и берём разбиение с максимальным Gain. Если даже лучший $\text{Gain} \leq 0$ — выигрыш не окупает штраф $\gamma$ за новый лист, и вершину не делим.

**Пример, $\lambda = 1$**




$$\text{Gain} = \frac{1}{2}(8.33 + 0.2 - 5.14) - \gamma \approx 1.7 - \gamma$$

$\gamma = 1$: $\text{Gain} \approx 0.7 > 0$ → **делим**

$\gamma = 2$: $\text{Gain} \approx -0.3$ → **не делим**

# XGBoost: основные особенности

## Вторая производная

Базовый алгоритм приближает направление, посчитанное с учётом второй производной функции потерь

## Регуляризация

В функционал добавлены штрафы за число листьев ($\gamma$) и за величину значений в листьях ($\lambda$ и $\alpha$ — L2 и L1)

## Выбор разбиения

Выигрыш от разбиения $\mathrm{Gain}$ считается прямо по суммам $S$ и $H$ в листьях — дерево строится под тот же функционал, который минимизируем

## Останов

Вершину делим, только если $\mathrm{Gain} > 0$: выигрыш должен окупить штраф $\gamma$ за новый лист

# XGBoost: инженерные приёмы

## Пропуски в данных

Sparsity-aware split: для каждого разбиения выучивается направление по умолчанию, куда отправлять объекты с пропуском

## Поиск порогов

Точный перебор (exact) или приближённый: кандидаты-квантили, взвешенные по $h_i$ (weighted quantile sketch), и гистограммный режим tree_method="hist"

## Параллельность

Данные хранятся отсортированными блоками по столбцам — поиск лучших разбиений по признакам идёт параллельно; есть обучение на GPU

## Подвыборки

subsample — по объектам, colsample_bytree / bylevel / bynode — по признакам, как в случайном лесе

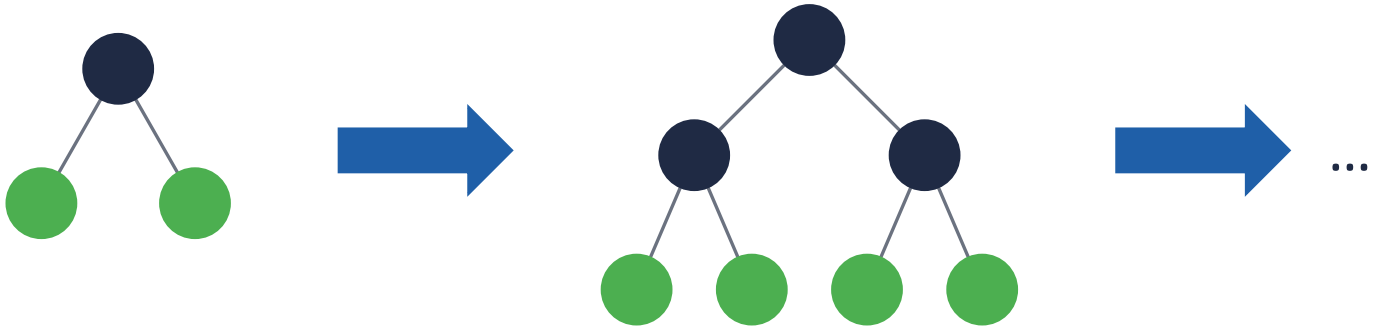
В новых версиях есть встроенная поддержка категориальных признаков (enable_categorical=True), но исторически их нужно было кодировать вручную.

# XGBoost: ключевые гиперпараметры

| Параметр | По умолчанию | Что делает |
|---|---|---|
| `n_estimators` | 100 | число деревьев; подбирать вместе с learning_rate и ранней остановкой |
| `learning_rate (eta)` | 0.3 | темп обучения $\eta$; меньше — качественнее, но нужно больше деревьев |
| `max_depth` | 6 | глубина дерева — главный рычаг сложности |
| `min_child_weight` | 1 | минимальная сумма $h_i$ в листе |
| `gamma (min_split_loss)` | 0 | $\gamma$: минимальный $\mathrm{Gain}$ для разбиения |
| `reg_lambda / reg_alpha` | 1 / 0 | L2- и L1-регуляризация значений в листьях |
| `subsample` | 1 | доля объектов для каждого дерева |
| `colsample_bytree` | 1 | доля признаков для каждого дерева |
| `tree_method` | hist | алгоритм поиска разбиений: exact / approx / hist |

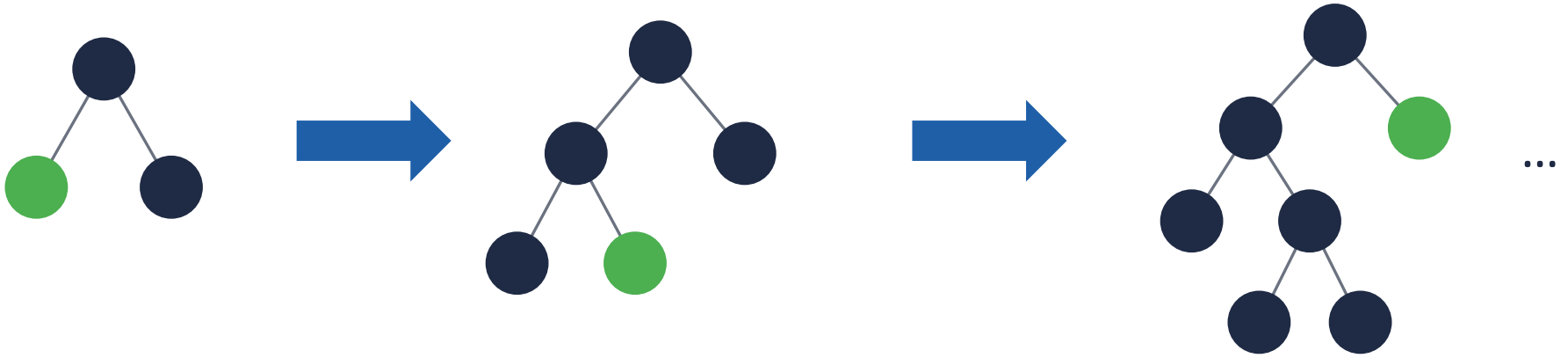
# LightGBM: рост дерева по листьям

**Level-wise: по уровням (XGBoost, классический GB)**



Каждый шаг делит лист с максимальным выигрышем — точнее решаем задачу оптимизации

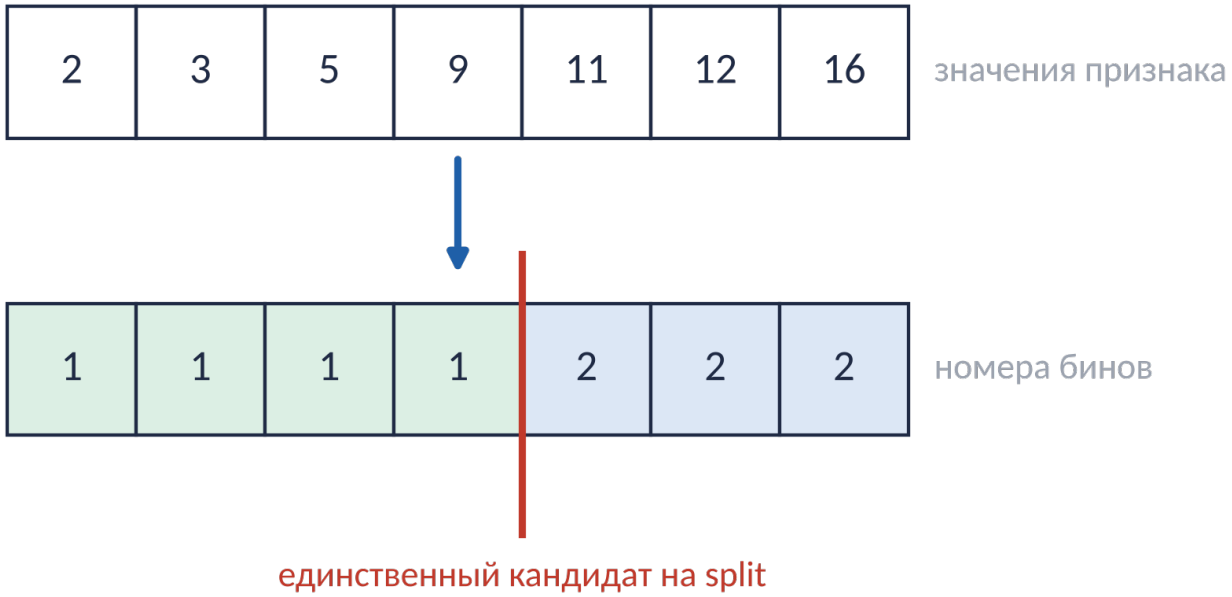
**Leaf-wise: по одному листу (LightGBM)**



Зелёный — лист, который разбивается на следующем шаге

# LightGBM: гистограммы признаков

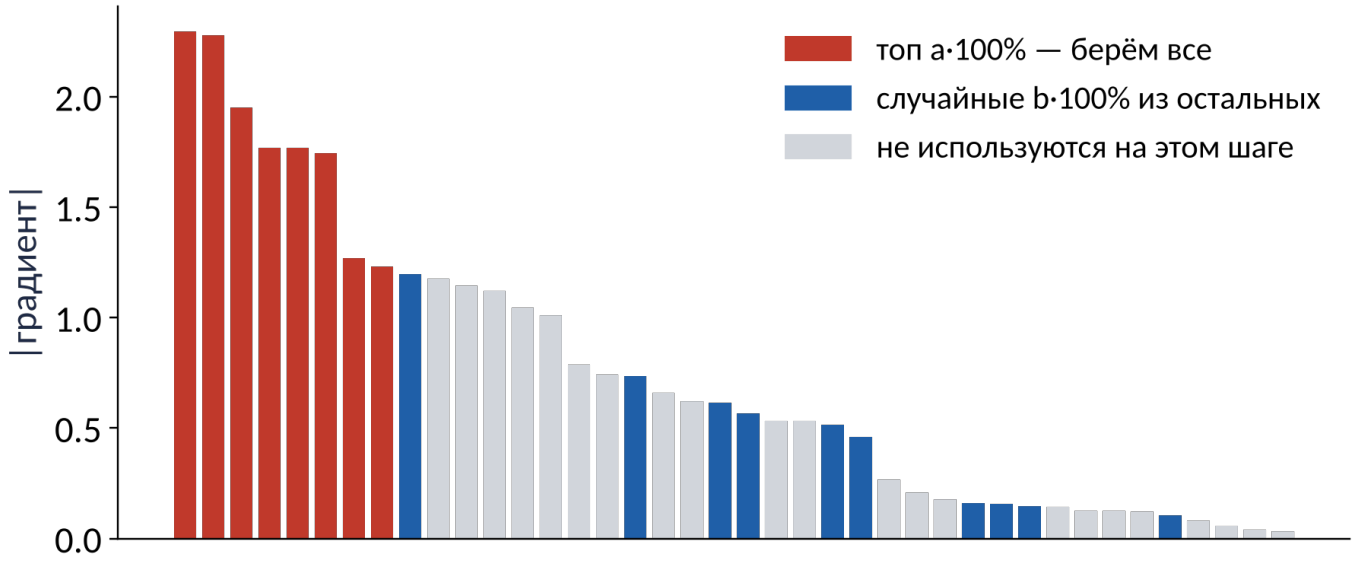
Ускорение построения деревьев за счёт бинаризации: значения признака заранее раскладываются по корзинам (max_bin = 255).




- Порог ищем не среди всех значений, а среди границ бинов: $O(\text{число бинов})$ вместо $O(\text{число объектов})$
- Для каждого бина копим суммы $s_i$ и $h_i$ — этого хватает для Gain

# LightGBM: GOSS — отбор объектов по градиенту

Gradient-based One-Side Sampling: объекты с маленьким $|s_i|$ уже хорошо предсказаны и мало влияют на выбор разбиения.




## Как работает

1. Сортируем объекты по $|s_i|$
2. Берём топ $a \cdot 100\%$ объектов
3. Из остальных случайно берём $b \cdot 100\%$
4. Их вклад умножаем на вес, чтобы не исказить распределение:

$$\text{вес} = \frac{1-a}{b}$$

Обучение на заметно меньшей выборке почти без потери точности. В новых версиях: data_sample_strategy="goss".

# LightGBM: EFB — склейка разреженных признаков

Exclusive Feature Bundling: в разреженных данных многие признаки почти никогда не бывают ненулевыми одновременно (например, после one-hot).

| $A$ | $B$ | **пучок** |
|---|---|---|
| 0 | 2 | 5 |
| 3 | 0 | 3 |
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 4 | 7 |

$B$ сдвигаем на $\max(A) = 3$, чтобы диапазоны не пересекались

- **Взаимоисключающие** признаки объединяем в один «пучок» со сдвигом значений — разбиение по пучку эквивалентно разбиению по исходным признакам
- **Поиск пучков** сводится к раскраске графа конфликтов; решается жадно, допускается небольшая доля конфликтов
- **Эффект:** число признаков резко падает → гистограммы строятся быстрее

# LightGBM: категориальные признаки

- LightGBM разбивает значения категориального признака на два подмножества в каждой вершине дерева, находя наилучшее разбиение
- Если у признака $k$ различных значений, то возможных разбиений $2^{k-1} - 1$ — полный перебор невозможен

**Трюк:** сортируем категории по накопленной статистике градиентов, после чего ищем порог, как для числового признака. Это даёт оптимальное разбиение за $O(k \log k)$ операций.

сортируем категории по $\frac{S_c}{H_c + \lambda} = \frac{\sum_{x_i \in c} s_i}{\sum_{x_i \in c} h_i + \lambda}$

- Признак нужно передать как categorical_feature (целые коды или pandas category)
- Для редких категорий легко переобучиться: помогают min_data_per_group, cat_smooth, cat_l2; при $k \leq 4$ (max_cat_to_onehot) используется one-vs-rest

# LightGBM: ключевые гиперпараметры

| Параметр | По умолчанию | Что делает |
|---|---|---|
| `num_leaves` | 31 | главный параметр сложности при leaf-wise росте; держите $< 2^d$, где $d$ = max_depth |
| `max_depth` | −1 | ограничение глубины (по умолчанию нет) |
| `min_data_in_leaf` | 20 | минимум объектов в листе — защита от переобучения |
| `learning_rate` | 0.1 | темп обучения $\eta$ |
| `n_estimators (num_iterations)` | 100 | число деревьев; используйте раннюю остановку |
| `feature_fraction` | 1.0 | доля признаков для каждого дерева |
| `bagging_fraction + bagging_freq` | 1.0 / 0 | подвыборка объектов и как часто её обновлять |
| `lambda_l1 / lambda_l2` | 0 / 0 | регуляризация значений в листьях |
| `max_bin` | 255 | число бинов в гистограммах |

# CatBoost

CatBoost — реализация градиентного бустинга от Яндекса. Главное отличие — умеет обрабатывать категориальные признаки «из коробки» и бороться со смещением, которое возникает при их кодировании.

### Симметричные деревья

одинаковое разбиение на всём уровне

### Категориальные признаки

статистики по перестановкам и комбинации признаков

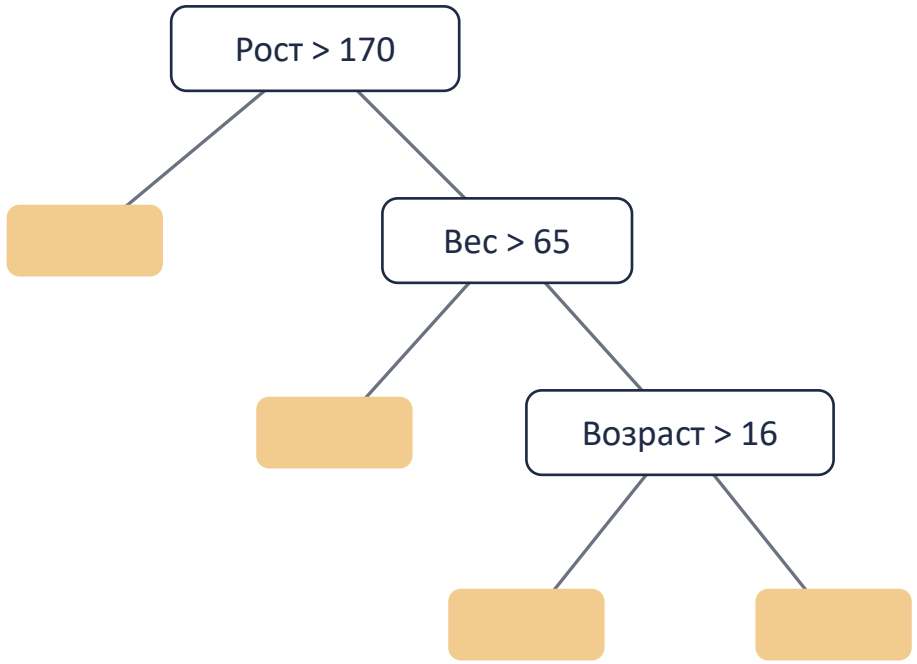
### Ordered boosting

динамический бустинг без утечки таргета

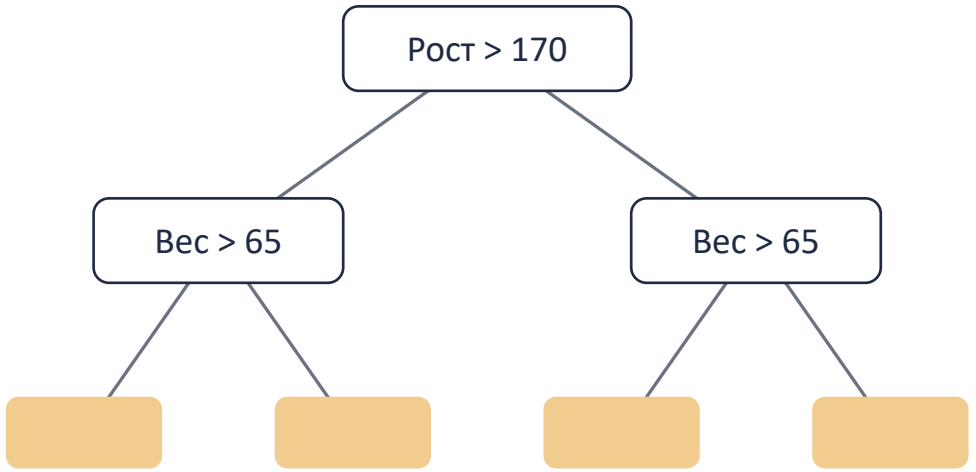
github.com/catboost/catboost

# CatBoost: симметричные деревья

**Обычное дерево**




**Симметричное (oblivious) дерево**




- На каждом уровне одно и то же условие → номер листа = двоичный код из $d$ ответов: очень быстрое применение модели
- Простая структура работает как регуляризация и меньше переобучается; глубина по умолчанию — 6

# CatBoost: категориальные признаки

Для кодирования категориальных признаков используется набор методов:

**One-hot**

для признаков с малым числом значений (one_hot_max_size)

**Счётчики**

статистики по таргету и частоты — без и с использованием таргета

**Перестановки**

статистика для объекта считается только по «прошлому»

**Комбинации**

жадно строит пары/тройки категорий

**Проблема наивного target encoding:** статистика по всей выборке включает таргет самого объекта → утечка таргета и сдвиг распределения признака между обучением и тестом.

$$\mathrm{TS}(x_i) = \frac{\sum_j [x_j = x_i]\, y_j}{\sum_j [x_j = x_i]}$$

# CatBoost: статистики по перестановкам

Перемешиваем объекты случайной перестановкой $\sigma$ и считаем статистику только по объектам, стоящим раньше:

| | категория | y |
|---|---|---|
| прошлое | SDE | 1 |
| прошлое | SDE | 1 |
| прошлое | SDE | 0 |
| прошлое | PR | 1 |
| **объект $i$** | **SDE** | **?** |
| будущее | PR | — |

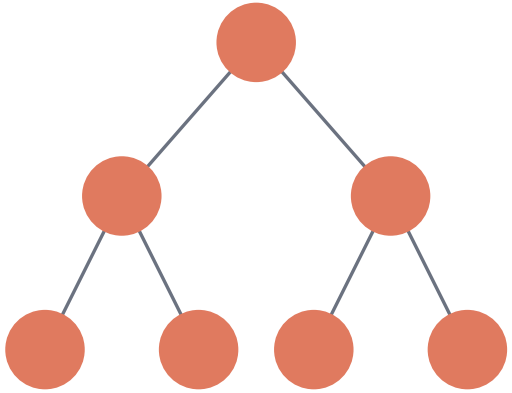
$$\mathrm{TS}(x_{\sigma_k}) = \frac{\sum_{j<k}[x_{\sigma_j} = x_{\sigma_k}]\, y_{\sigma_j} + \alpha\, p}{\sum_{j<k}[x_{\sigma_j} = x_{\sigma_k}] + \alpha}$$

$$\mathrm{TS}(i) = \frac{1+1+0}{3} = \frac{2}{3} \qquad \text{с приором } (\alpha = 1,\ p = 0.5): \quad \frac{2+0.5}{3+1} = 0.625$$

$p$ — априорное значение (например, среднее $y$), $\alpha$ — его вес. Чтобы уменьшить дисперсию, используют несколько перестановок.

# CatBoost: ordered (динамический) бустинг

Проблема: в обычном бустинге остатки для объекта считаются моделью, которая уже обучалась на этом же объекте → оценка градиента смещена (prediction shift).



$$\text{leafValue}(x_i) = \frac{\sum_{j<i,\, x_j \in R} g\big(a(x_j),\, y_j\big)}{\#\{\, j < i :\; x_j \in R \,\}}$$

Значение в листе для объекта считается только по объектам, которые раньше него в перестановке — как будто модель его ещё не видела. Тот же принцип, что и для статистик по категориям.

# CatBoost: бонусы реализации и гиперпараметры

- Поддержка пропусков в числовых признаках
- Хорошее качество даже без подбора параметров
- Быстрое применение модели, обучение на GPU
- Удобные методы: детектор переобучения, метрики, кросс-валидация, важность признаков, работа с текстами и эмбеддингами

| Параметр | По умолчанию |
|---|---|
| `iterations` | 1000 |
| `learning_rate` | авто |
| `depth` | 6 |
| `l2_leaf_reg` | 3 |
| `one_hot_max_size` | 2 |
| `cat_features` | — |
| `early_stopping_rounds` | — |

learning_rate по умолчанию подбирается автоматически по размеру данных и числу итераций.

# Сравнение реализаций

| | XGBoost | LightGBM | CatBoost |
|---|---|---|---|
| **Рост дерева** | по уровням (есть и lossguide) | по листьям | симметричные деревья |
| **Поиск разбиений** | exact / квантили / hist | гистограммы, GOSS, EFB | гистограммы (бинаризация) |
| **Категориальные** | встроенно в новых версиях | встроенно, сортировка по градиентам | ordered TS + комбинации |
| **Пропуски** | выученное направление | выученное направление | обрабатываются как min/max |
| **Скорость обучения** | средняя | обычно самая высокая | средняя, быстро на GPU |
| **Без настройки** | нужно настраивать | нужно настраивать | хорошие значения по умолчанию |

# Что выбрать на практике

### Много категориальных признаков

CatBoost: меньше ручного кодирования и утечек таргета

### Большие данные, важна скорость

LightGBM: гистограммы, GOSS и EFB дают самое быстрое обучение

### Нужен контроль и стабильность

XGBoost: зрелая библиотека, много вариантов регуляризации

### Общий рецепт настройки

1) Зафиксировать небольшой learning_rate (0.03–0.1) и включить раннюю остановку по валидации
2) Подобрать сложность деревьев: max_depth / num_leaves / min_data_in_leaf
3) Добавить подвыборки по объектам и признакам и регуляризацию $\lambda$

# Итоги

1. **Бэггинг:** уменьшает разброс, бустинг — смещение

2. **Бустинг:** последовательно обучает деревья на ошибки композиции

3. **Градиентный бустинг:** обучает дерево на антиградиент любой дифференцируемой функции потерь

4. **Регуляризация:** неглубокие деревья, малый $\eta$, подвыборки, ранняя остановка

5. **Реализации:** XGBoost — вторая производная и регуляризация; LightGBM — скорость; CatBoost — категориальные признаки